

Ema Lovrić, 22.5.2026.

Assignment 8: Task automation with bash and PowerShell

<https://github.com/elovric13/eutech/tree/task-automation-with-bash-and-powershell-elovric13>

1. Create a bash script

A Bash script was written to automate three administrative tasks on a Linux system: adding users from a text file, creating a compressed backup of a directory, and logging all actions with timestamps. The script is located at `/root/bash_script.sh` and was made executable with `chmod +x`.

- The script reads a file called `users.txt` and for each username it checks whether the user already exists. If the user does not exist, it creates the account with `useradd -m` and sets a default password. If the user already exists, it skips it and logs a message.

```
ubuntu26@ubuntu26:~$ sudo cat /root/users.txt
alice
bob
charlie
```

Figure 1: `users.txt` file with test usernames

```
[2026-05-22 14:45:59] Starting user creation.
[2026-05-22 14:46:00] User alice created
[2026-05-22 14:46:01] User bob created
[2026-05-22 14:46:01] User charlie created
[2026-05-22 14:46:01] User creation finished.
```

Figure 2: Script output showing users being created

- The script creates a `.tar.gz` archive of the `/home` directory and saves it to `/backup` with a filename that includes the current date and time.

```
ubuntu26@ubuntu26:~$ ls /backup
backup_2026-05-22_14-45-59.tar.gz
```

Figure 3: Backup file created in `/backup`

- Every action the script performs is written to `/var/log/bash_script.log` with a timestamp in the format `[YYYY-MM-DD HH:MM:SS]`.

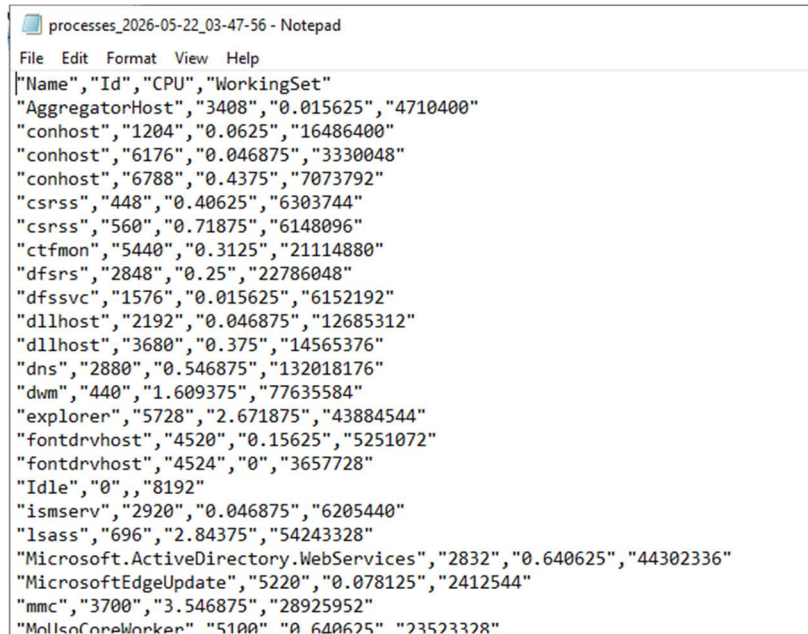
```
ubuntu26@ubuntu26:~$ cat /var/log/bash_script.log
[2026-05-22 14:45:59] Script started
[2026-05-22 14:45:59] Starting user creation.
[2026-05-22 14:46:00] User alice created
[2026-05-22 14:46:01] User bob created
[2026-05-22 14:46:01] User charlie created
[2026-05-22 14:46:01] User creation finished.
[2026-05-22 14:46:01] Starting backup
[2026-05-22 14:46:23] Backup created successfully: /backup/backup_2026-05-22_14-45-59.tar.gz
[2026-05-22 14:46:23] Backup finished
```

Figure 4: Log file contents showing timestamped entries

2. Create a PowerShell script

A PowerShell script was written to automate three administrative tasks on a Windows system: exporting active processes to a .csv file, performing a folder backup as a .zip archive, and checking CPU usage with an alert if it exceeds 80%. The script is located at C:\Scripts\ps_script.ps1

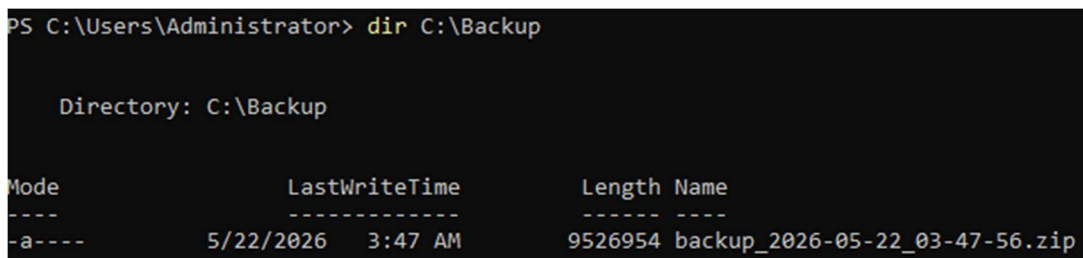
The script uses Get-Process to retrieve all running processes and selects the Name, Id, CPU, and WorkingSet fields. The output is exported using Export-Csv to C:\Logs\ with a timestamped filename.



```
processes_2026-05-22_03-47-56 - Notepad
File Edit Format View Help
"Name","Id","CPU","WorkingSet"
"AggregatorHost","3408","0.015625","4710400"
"conhost","1204","0.0625","16486400"
"conhost","6176","0.046875","3330048"
"conhost","6788","0.4375","7073792"
"csrss","448","0.40625","6303744"
"csrss","560","0.71875","6148096"
"ctfmon","5440","0.3125","21114880"
"dfsrs","2848","0.25","22786048"
"dfssvc","1576","0.015625","6152192"
"dllhost","2192","0.046875","12685312"
"dllhost","3680","0.375","14565376"
"dns","2880","0.546875","132018176"
"dwm","440","1.609375","77635584"
"explorer","5728","2.671875","43884544"
"fontdrvhost","4520","0.15625","5251072"
"fontdrvhost","4524","0","3657728"
"Idle","0","0","8192"
"ismserv","2920","0.046875","6205440"
"lsass","696","2.84375","54243328"
"Microsoft.ActiveDirectory.WebServices","2832","0.640625","44302336"
"MicrosoftEdgeUpdate","5220","0.078125","2412544"
"mmc","3700","3.546875","28925952"
"MollsoCoreWorker","5100","0.640625","23523328"
```

Figure 5: .csv file with exported process list

The script compresses the C:\Users directory into a .zip archive using Compress-Archive and saves it to C:\Backup\ with a timestamped filename.



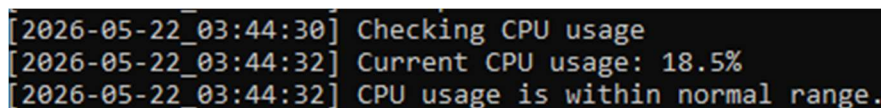
```
PS C:\Users\Administrator> dir C:\Backup

Directory: C:\Backup

Mode                LastWriteTime         Length Name
----                -
-a----          5/22/2026   3:47 AM         9526954 backup_2026-05-22_03-47-56.zip
```

Figure 6: Backup .zip file

The script queries the current CPU load using Get-CimInstance Win32_Processor. If the average load exceeds 80%, it logs an alert message and displays a popup warning.



```
[2026-05-22_03:44:30] Checking CPU usage
[2026-05-22_03:44:32] Current CPU usage: 18.5%
[2026-05-22_03:44:32] CPU usage is within normal range.
```

Figure 7: Log showing CPU usage reading

```

[2026-05-22_03:47:56] Script started
[2026-05-22_03:47:56] Starting process export
[2026-05-22_03:47:56] Process list exported to: C:\Logs\processes_2026-05-22_03-47-56.csv
[2026-05-22_03:47:56] Process export finished.
[2026-05-22_03:47:56] Starting backup
[2026-05-22_03:47:58] Backup created successfully: C:\Backup\backup_2026-05-22_03-47-56.zip
[2026-05-22_03:47:58] Backup finished
[2026-05-22_03:47:58] Checking CPU usage
[2026-05-22_03:48:00] Current CPU usage: 10.5%
[2026-05-22_03:48:00] CPU usage is within normal range.
[2026-05-22_03:48:00] Script completed

```

Figure 8: Log file contents showing timestamped entries

3. Schedule execution with cron (Linux) and Task Scheduler (Windows)

- The Bash script was scheduled using crontab -e. The cron job was configured to run the script every day at 2:00 AM.

```
0 2 * * * /bin/bash /root/bash_script.sh
```

Figure 9: crontab -e showing the scheduled job

- The PowerShell script was scheduled using Windows Task Scheduler. A new task was created with a daily trigger set to run at 2:00 AM. The action was set to run powershell.exe with the argument -File C:\Scripts\ps_script.ps1.

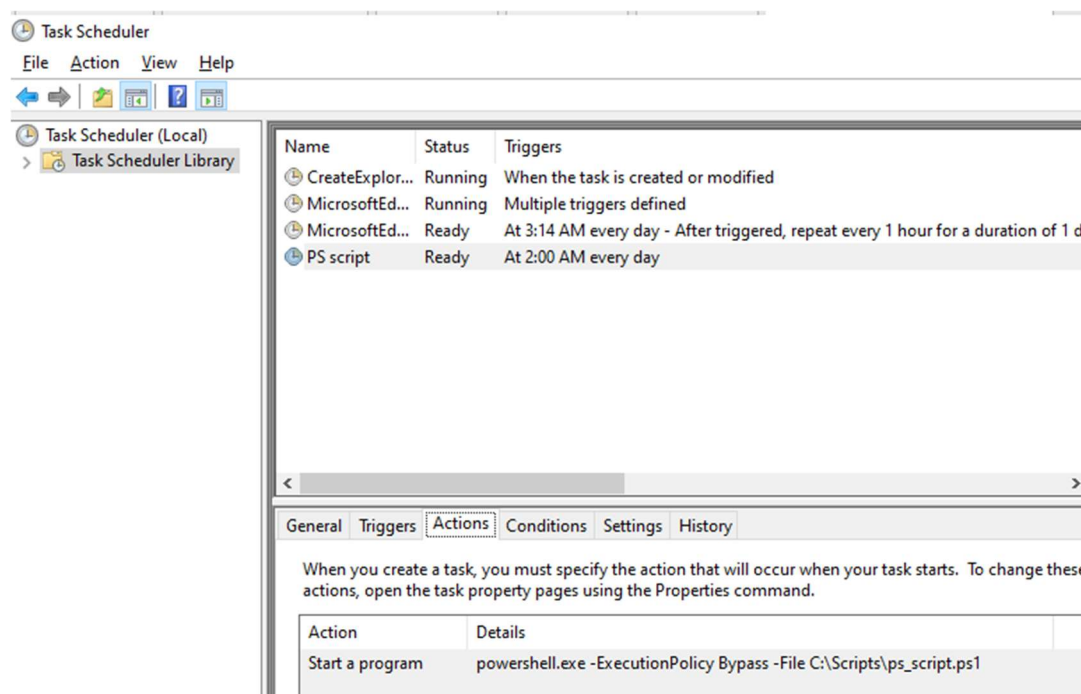


Figure 10: Task Scheduler showing the created task